



Analisis Mendalam Blockchain SUI

Arsitektur, Deployment, dan Pengujian Jaringan dalam Lingkungan Kontainer

Petrus Tyang Agung Rosario
Peneliti Blockchain Independen
petrus.rosario@example.com

Juli 2026

Abstract

Blockchain Sui mewakili pergeseran mendasar dalam desain jaringan terdesentralisasi dengan memperkenalkan model data berpusat pada objek (object-centric) dan memisahkan diseminasi transaksi dari pengurutan konsensus global. Laporan riset ini mengeksplorasi inovasi arsitektur utama Sui, termasuk bahasa pemrograman Sui Move, eksekusi paralel, dan mesin konsensus Mysticeti. Kami mendokumentasikan deployment jaringan multi-node lokal Sui dalam lingkungan kontainer dan melakukan pengujian performa (TPS dan latensi di bawah beban stres), chaos testing (ketahanan kegagalan node), serta evaluasi smart contract. Temuan kami menunjukkan kemampuan praktis Sui dalam mencapai finalitas sub-detik dan throughput konkurensi yang tinggi, sekaligus mengidentifikasi keterbatasan operasional dan tantangan tata kelola terdesentralisasi.

Daftar Isi

1	Pendahuluan	3
1.1	Motivasi dan Tujuan Penelitian	3
1.2	Pertanyaan Penelitian	3
1.3	Struktur Makalah	4
2	Gambaran Teknis Arsitektur SUI	5
2.1	Model Data Berpusat pada Objek (Object-Centric)	5
2.2	Mesin Konsensus: Narwhal, Bullshark, dan Mysticeti	5
2.2.1	Narwhal: Mempool DAG	5
2.2.2	Bullshark dan Mysticeti: Mesin Pengurut (Ordering)	6
2.3	Bahasa Pemrograman Sui Move	6
2.3.1	Keamanan Berorientasi Sumber Daya	6
2.3.2	Kekhasan Sui Move	7
3	Metodologi Penelitian	8
3.1	Desain Eksperimen	8
3.2	Variabel Eksperimen	8
3.3	Skenario Evaluasi	8
4	Implementasi Sistem dan Deployment	10
4.1	Setup Lingkungan	10
4.2	Desain Skrip Otomatisasi	10
4.3	Topologi Jaringan Lokal	10
4.4	Setup Smart Contract Move	11
5	Evaluasi Eksperimental dan Hasil	12
5.1	S1: Aliran Transaksi Dasar	12
5.2	S2 & S3: Deployment dan Interaksi Smart Contract	12
5.3	S4 & S5: Benchmarking Throughput (TPS) dan Latensi	12
5.4	Kendala Penguncian Koin Gas dan Kegagalan Konkurensi	12
5.5	S6 & S7: Analisis Persaingan Objek (Contention)	13
5.6	S8: Kegagalan dan Pemulihan Node (Chaos Testing)	14
5.7	S9: Pemanfaatan Sumber Daya Sistem	14
6	Diskusi dan Analisis Perbandingan	15
6.1	Implikasi Arsitektural dari Paralelisme	15
6.2	Analisis Perbandingan: Sui vs. Cosmos SDK	15
6.3	Desentralisasi dan Persyaratan Perangkat Keras	16
7	Kesimpulan dan Saran	17
7.1	Ringkasan Temuan	17
7.2	Saran dan Pekerjaan di Masa Depan	17
A	Lampiran	19
A.1	Kode Smart Contract Sui Move (hello_sui.move)	19
A.2	Skrip Benchmarking Konkurensi (benchmark-tps.sh)	20
A.3	Skrip Pengujian Chaos (chaos-test.sh)	22

1 Pendahuluan

Teknologi blockchain telah bertransisi dari jaringan uang elektronik peer-to-peer sederhana menjadi platform komputasi terdesentralisasi kompleks yang menghosting aplikasi terdistribusi (dApps). Blockchain Layer 1 tradisional, seperti Ethereum, mengandalkan model eksekusi sekuensial dan penyimpanan key-value global di mana transisi status diproses dalam urutan linier. Meskipun hal ini menjamin pengurutan deterministik yang ketat, hal ini memperkenalkan hambatan skalabilitas yang parah (sering disebut sebagai “blockchain trilemma”). Di bawah volume transaksi yang tinggi, throughput jaringan menurun, latensi validasi transaksi melonjak, dan biaya gas berfluktuasi secara dramatis.

Untuk memecahkan batasan skalabilitas ini, arsitektur Layer 1 yang lebih baru telah muncul. Di antaranya, blockchain Sui, yang dikembangkan oleh Mysten Labs, mengusulkan pergeseran paradigma yang mendasar. Alih-alih memperlakukan status ledger sebagai ruang akun monolitik tunggal, Sui memperkenalkan *model data berpusat pada objek (object-centric)*. Di bawah desain ini, setiap aset, koin, smart contract, dan kredensial adalah “objek” kelas satu yang dapat dialamatkan dengan tipe kepemilikan yang jelas. Hal ini memungkinkan Sui untuk secara native mengidentifikasi dan mengeksekusi transaksi independen secara paralel, melewati hambatan pengurutan konsensus global tradisional untuk aset yang tidak saling berkonflik.

Selain itu, Sui memisahkan diseminasi transaksi dari pengurutan. Sui menggunakan mempool berbasis Directed Acyclic Graph (DAG), Narwhal, yang dipasangkan dengan mesin konsensus (secara historis Bullshark, dan baru-baru ini Mysticeti). Mysticeti mengurangi latensi bolak-balik konsensus ke tingkat sub-detik, memungkinkan finalitas cepat untuk objek bersama (shared objects) sementara objek yang dimiliki (owned objects) dapat melewati konsensus sepenuhnya menggunakan mekanisme Byzantine Consistent Broadcast (BCB).

1.1 Motivasi dan Tujuan Penelitian

Meskipun klaim teoretis Sui sangat kuat—seperti mendukung ratusan ribu Transaksi Per Detik (TPS) dan finalitas sub-detik—masih ada kekurangan penelitian empiris independen yang menganalisis kinerja, stabilitas, dan persyaratan operasionalnya dalam lingkungan infrastruktur yang terkendali. Sebagian besar benchmark yang ada dirilis oleh Mysten Labs sendiri atau dilakukan di testnet yang disimulasikan.

Penelitian ini bertujuan untuk menjembatani kesenjangan ini dengan melakukan deployment jaringan Sui lokal di dalam lingkungan kontainer dan melakukan evaluasi komprehensif terhadap kinerja, pengalaman pengembang, dan ketahanan sistem. Tujuan utamanya adalah:

1. Menyelidiki dan mendokumentasikan inovasi arsitektur Sui (model berpusat pada objek, bahasa Move, dan konsensus berbasis DAG).
2. Melakukan deployment dan mendokumentasikan jaringan pengembangan lokal Sui multi-node yang dapat direplikasi di dalam kontainer.
3. Melakukan benchmark empiris throughput transaksi (TPS) dan latensi di bawah berbagai beban jaringan.
4. Menganalisis ketahanan jaringan melalui pengujian chaos (khususnya kegagalan node validator).
5. Mengevaluasi pengalaman pengembang dan jaminan keamanan bahasa pemrograman Sui Move.

1.2 Pertanyaan Penelitian

Untuk memandu studi empiris kami, kami merumuskan tiga Pertanyaan Penelitian (Research Questions - RQ) utama:

- RQ1:** *Bagaimana model data berpusat pada objek Sui diterjemahkan ke dalam throughput transaksi paralel dan latensi di bawah pengujian stres dalam jaringan lokal?*
- RQ2:** *Bagaimana perilaku ketahanan dan pemulihan mekanisme konsensus Sui (Mysticeti/Bullshark) ketika node validator mengalami kegagalan mendadak atau partisi jaringan?*
- RQ3:** *Bagaimana pengalaman pengembang dan keamanan kontrak Sui Move dibandingkan dengan pengembangan EVM/Solidity berbasis akun tradisional?*

1.3 Struktur Makalah

Sisa dari makalah ini terstruktur sebagai berikut. Bagian 2 memberikan gambaran teknis tentang arsitektur inti Sui, termasuk model datanya, mesin konsensus, dan bahasa pemrograman Move. Bagian 3 menjelaskan metodologi penelitian, variabel eksperimental, dan skenario pengujian. Bagian 4 merinci deployment terkontainerisasi dan konfigurasi jaringan lokal. Bagian 5 menyajikan hasil empiris dari benchmarking kami, eksekusi smart contract, dan chaos testing. Bagian 6 menganalisis implikasi dari temuan kami, membandingkannya dengan arsitektur platform lain (like Cosmos SDK), dan membahas masalah desentralisasi serta keamanan. Terakhir, Bagian 7 merangkum temuan penelitian dan menguraikan arah untuk pekerjaan di masa depan.

2 Gambaran Teknis Arsitektur SUI

Blockchain Sui memperkenalkan inovasi utama yang membedakannya dari jaringan berbasis akun tradisional seperti Ethereum atau Cosmos SDK. Memahami inovasi ini sangat penting untuk menganalisis kinerja dan keamanannya. Bagian ini merinci model data berpusat pada objek Sui, mekanisme konsensusnya, dan bahasa pemrograman Move.

2.1 Model Data Berpusat pada Objek (Object-Centric)

Pada blockchain tradisional (misalnya, Ethereum), status global adalah ledger monolitik yang memetakan alamat akun ke saldo masing-masing dan slot penyimpanan smart contract. Untuk mengeksekusi transaksi, jaringan harus mengunci status global, memperbarui saldo secara berurutan, dan membukanya. Hal ini memperkenalkan persaingan (contention) yang tinggi dan membatasi kecepatan eksekusi hanya pada satu thread.

Sui menggantikan paradigma berbasis akun ini dengan *model data berpusat pada objek* [1]. Di Sui, status global direpresentasikan sebagai kumpulan objek independen yang dapat dialamatkan, masing-masing diberikan ID Objek 32-byte yang unik secara global. Setiap objek memiliki serangkaian atribut, termasuk pemiliknya, versinya (sequence number), digest transaksi (hash dari transaksi terakhir yang memutasi objek tersebut), dan data binernya.

Objek Sui dikategorikan berdasarkan empat jenis kepemilikan:

1. **Address-Owned Objects (Objek Milik Alamat):** Dikendalikan oleh pasangan kunci kriptografi tertentu (misalnya, saldo koin individu atau NFT). Hanya pemilik yang dapat menandatangani transaksi untuk memutasi atau mentransfer objek tersebut.
2. **Shared Objects (Objek Bersama):** Dapat diakses oleh siapa saja. Banyak pengguna dapat membaca atau menulis ke objek bersama (misalnya, pool decentralized exchange, kontrak lelang, atau papan permainan bersama). Objek-objek ini memerlukan pengurutan konsensus global untuk mencegah race conditions.
3. **Immutable Objects (Objek Imutabel):** Aset baca-saja yang tidak dapat dimutasi, dihapus, atau ditransfer setelah pembuatan (misalnya, paket kode smart contract yang dideploy).
4. **Object-Owned Objects (Objek Milik Objek):** Dimiliki oleh objek lain, memungkinkan komposisi hierarkis (misalnya, objek karakter game yang memiliki objek pedang).

Karena objek milik alamat independen satu sama lain, transaksi yang melibatkan objek milik alamat yang berbeda dapat diproses secara paralel di beberapa core CPU, seperti yang ditunjukkan pada Gambar 1.

2.2 Mesin Konsensus: Narwhal, Bullshark, dan Mysticeti

Sui memisahkan diseminasi transaksi (mempool) dari pengurutan transaksi (konsensus) untuk memaksimalkan throughput jaringan [2].

2.2.1 Narwhal: Mempool DAG

Mempool tradisional menyiarkan transaksi secara tidak teratur, menyebabkan transmisi redundan dan hambatan CPU selama pengurutan. Narwhal memecahkan ini dengan mengatur batch transaksi ke dalam Directed Acyclic Graph (DAG). Validator mengusulkan batch transaksi dan merujuk ke batch yang diusulkan sebelumnya dari validator lain, membangun grafik kausal. Narwhal menjamin ketersediaan data, artinya setelah batch menjadi bagian dari DAG, semua validator jujur dijamin memiliki akses ke isinya.

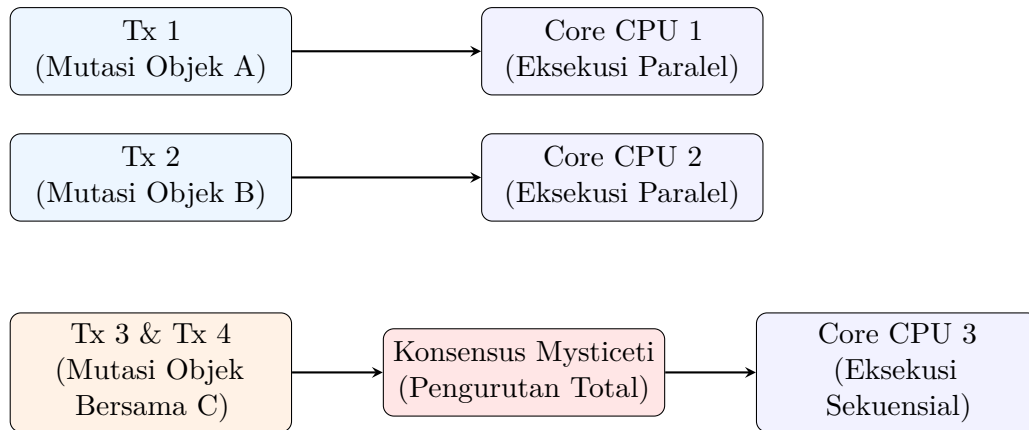


Figure 1: Alur Eksekusi Transaksi Paralel vs. Sekuensial Sui

2.2.2 Bullshark dan Mysticeti: Mesin Pengurut (Ordering)

Setelah Narwhal menetapkan DAG, mesin pengurut menavigasi grafik untuk menentukan urutan total transaksi tanpa memerlukan pesan voting tambahan.

- **Bullshark:** Mesin konsensus awal Sui. Ini mengurutkan DAG dalam putaran (waves), menunggu kondisi tertentu sebelum melakukan commit sub-graph. Meskipun sangat efisien dalam throughput, ia memperkenalkan penundaan latensi karena menunggu proposal leader di setiap wave.
- **Mysticeti:** Diperkenalkan oleh Mysten Labs pada tahun 2024 untuk menggantikan Bullshark [3]. Mysticeti mempertahankan struktur DAG tetapi mengoptimalkan aturan commit. Ia menggunakan pendekatan validator aktif-aktif di mana validator dapat melakukan commit sub-graph secara terus-menerus, melewati waktu tunggu leader. Ini memangkas latensi konsensus menjadi sekitar 400 milidetik.

Untuk transaksi sederhana (misalnya, mentransfer aset milik alamat), Sui melewati konsensus sepenuhnya. Sebagai gantinya, Sui menggunakan protokol Byzantine Consistent Broadcast (BCB) yang ringan. “Jalur cepat” (fast path) ini mencapai validasi instan karena hanya mengharuskan klien untuk mengumpulkan tanda tangan validator (sertifikat) yang mengonfirmasi kebenaran kepemilikan dan versi.

2.3 Bahasa Pemrograman Sui Move

Sui Move adalah adaptasi dari bahasa Move yang awalnya dikembangkan oleh Meta untuk proyek Diem [4]. Bahasa ini dirancang khusus untuk manajemen sumber daya (resource) yang aman.

2.3.1 Keamanan Berorientasi Sumber Daya

Berbeda dengan Solidity, yang melacak aset sebagai saldo di dalam pemetaan penyimpanan tingkat kontrak, Move memperlakukan aset sebagai objek fisik (sumber daya) dengan aturan ketat yang diterapkan oleh compiler:

- **Tidak Ada Penyalinan atau Penghapusan Implisit:** Objek yang mewakili token atau NFT tidak dapat disalin (yang akan menyebabkan pengeluaran ganda/double spending) atau dihapus secara implisit (yang akan menghilangkan aset). Objek tersebut harus ditransfer, dibakar, atau disimpan secara eksplisit.

- **Keamanan Tipe yang Ketat (Strict Type Safety):** Verifikator bytecode Move memeriksa tipe dan keamanan kepemilikan secara off-chain sebelum eksekusi kontrak, mencegah serangan reentrancy, integer overflow, dan akses tidak sah sejak awal desain.

2.3.2 Kekhasan Sui Move

Sementara Move standar menyimpan sumber daya di dalam penyimpanan akun, Sui Move menyimpan sumber daya sebagai objek global independen. Kontrak Sui Move adalah objek paket berisi modul yang mendefinisikan tipe struct kustom (mewakili objek) dan fungsi entry untuk menginstansiasi, memutasi, atau mentransfernya. Integrasi struktural dengan model objek blockchain ini memungkinkan kontrak Sui Move berjalan dengan paralelisme maksimum.

3 Metodologi Penelitian

Bagian ini menjelaskan metodologi, desain eksperimen, dan metrik yang digunakan untuk mengevaluasi kinerja dan ketahanan blockchain Sui.

3.1 Desain Eksperimen

Untuk melakukan analisis empiris yang terkontrol, kami melakukan deployment jaringan Sui lokal pada server host tunggal menggunakan virtualisasi. Jaringan dikonfigurasi dengan empat node validator dan satu full node lokal, mereplikasi konfigurasi pengujian default yang direkomendasikan untuk pengembangan lokal Sui. Setup ini mengisolasi blockchain dari latensi internet eksternal dan kemacetan jaringan publik, memastikan bahwa hasilnya mencerminkan kemampuan intrinsik dari perangkat lunak dan alokasi perangkat keras.

3.2 Variabel Eksperimen

Kami mendefinisikan tiga kategori variabel eksperimen untuk benchmarking kami:

1. Variabel Bebas (Independent Variables):

- **Laju Beban Transaksi:** Laju di mana transaksi disuntikkan ke dalam mempool (berkisar antara 100 transaksi/detik hingga 5.000 transaksi/detik).
- **Tingkat Konkurensi:** Jumlah proses klien paralel yang mengirimkan transaksi secara bersamaan.
- **Status Validator:** Status node validator (aktif, offline, atau mengalami simulasi partisi jaringan).

2. Variabel Terikat (Dependent Variables):

- **Throughput (TPS):** Jumlah transaksi yang berhasil dimasukkan ke dalam checkpoint per detik.
- **Latensi:** Waktu yang berlalu sejak transaksi ditandatangani dan disiarkan oleh klien hingga dimasukkan ke dalam checkpoint yang telah berkomitmen.
- **Pemanfaatan Sumber Daya Sistem:** Penggunaan CPU, RAM, disk, dan jaringan di semua node.

3. Variabel Kontrol (Control Variables):

- **Spesifikasi Perangkat Keras:** Alokasi tetap 8 core CPU (Intel Xeon E5-2697 v2 @ 2.70GHz), RAM 31 GB, dan penyimpanan SSD.
- **Konfigurasi Perangkat Lunak:** Versi devnet Sui, parameter konfigurasi default (misalnya, batas waktu blok, batas waktu konsensus, batas mempool).

3.3 Skenario Evaluasi

Kami merancang sembilan skenario pengujian yang berbeda untuk menjawab pertanyaan penelitian kami:

S1: Aliran Transaksi Dasar. Mengirimkan transfer objek milik alamat sederhana antara dua akun untuk memverifikasi bahwa eksekusi “jalur cepat” melewati konsensus global.

S2: Deployment Kontrak Move. Mengompilasi dan mendeploy smart contract Move kustom untuk mengevaluasi keamanan kompiler dan biaya gas deployment.

- S3: Interaksi Smart Contract.** Mengeksekusi transaksi baca-saja dan tulis-saja yang memanggil fungsi entry di kontrak Move yang dideploy, mengukur biaya gas dan waktu respons.
- S4: Benchmarking Throughput (TPS).** Meningkatkan laju penyuntikan transaksi secara bertahap untuk menentukan ambang batas puncak TPS dari jaringan lokal.
- S5: Profiling Latensi Transaksi.** Mengukur rata-rata latensi finalitas transaksi di berbagai tingkat throughput.
- S6: Transfer Kepemilikan Objek.** Melakukan benchmark kinerja memutasi objek milik alamat dibandingkan dengan mentransfer objek bersama.
- S7: Persaingan Objek Bersama (Contention).** Mengirimkan transaksi berkonflik yang menargetkan objek bersama yang sama (misalnya, memperbarui counter secara bersamaan) untuk mengukur dampak persaingan pada throughput.
- S8: Kegagalan dan Pemulihan Node (Chaos Testing).** Menghentikan node validator selama benchmarking aktif untuk menganalisis bagaimana konsensus Mysticeti/Bullshark menangani kegagalan node dan mengukur waktu pemulihan setelah restart.
- S9: Pemantauan Sumber Daya Sistem.** Mengumpulkan metrik kinerja sistem host (CPU, RAM, Disk I/O) di bawah kondisi beban puncak.

4 Implementasi Sistem dan Deployment

Bagian ini merinci setup fisik jaringan pengembangan lokal SUI dan implementasi skrip otomatisasi kami.

4.1 Setup Lingkungan

Lingkungan eksperimen dideploy pada server Ubuntu 22.04 LTS. Binari SUI dikelola menggunakan `suiup`, yang memungkinkan peralihan antara konfigurasi devnet, testnet, dan localnet.

Untuk mendeploy jaringan, kami mengeksekusi serangkaian skrip otomatisasi yang menginisialisasi database, mengonfigurasi genesis lokal, dan menjalankan set validator.

4.2 Desain Skrip Otomatisasi

Kami mengimplementasikan dua skrip deployment utama untuk mengelola siklus hidup jaringan lokal:

1. **install-sui.sh:** Memeriksa prasyarat (Rust, cargo, Git, dan build-essential), mengunduh manajer `suiup`, menginstal versi SUI yang ditentukan, dan mengonfigurasi variabel path lingkungan.
2. **setup-localnet.sh:** Menginisialisasi konfigurasi klien SUI, menjalankan set validator testnet lokal, menyiapkan endpoint RPC lokal di `http://127.0.0.1:9000`, menjalankan layanan faucet lokal di `http://127.0.0.1:9123`, dan meminta token tes SUI pertama.

4.3 Topologi Jaringan Lokal

Secara default, menjalankan `sui start` akan memulai proses lokal berikut:

- **4 Node Validator:** Menjalankan protokol konsensus (Mysticeti/Bullshark dan Narwhal DAG).
- **1 Full Node:** Melayani endpoint RPC JSON-RPC pada port 9000.
- **1 Layanan Faucet:** Berjalan pada port 9123, memungkinkan pendanaan alamat uji secara programatis.

Struktur file dari direktori kerja node diatur di bawah folder penelitian kami, mengisolasi log, konfigurasi, dan penyimpanan data. Listing 1 menunjukkan cuplikan dari skrip `setup-localnet.sh` kami.

```

1 #!/bin/bash
2 # Setup SUI Local Network
3 BASE_DIR="/home/ubuntu/lxc-manager/projects/sui-research"
4 LOG_DIR="$BASE_DIR/data/logs"
5 mkdir -p $LOG_DIR
6
7 echo "Starting SUI Local Network..."
8 # Start the local network in the background using sui start
9 sui start \
10 --network.config "$DATA_DIR" \
11 --force-regenesis \
12 --with-faucet=0.0.0.0:9123 \
13 --fullnode-rpc-port 9000 \
14 --epoch-duration-ms 60000 \
15 > "$LOG_DIR/validator.log" 2>&1 &
16
17 echo $! > "$BASE_DIR/data/validator.pid"
18 sleep 5

```

```
19  
20 # Set client to point to localnet  
21 sui client new-env --alias localnet --rpc http://127.0.0.1:9000  
22 sui client switch --env localnet
```

Listing 1: Cuplikan skrip setup-localnet.sh

4.4 Setup Smart Contract Move

Untuk mengeksekusi dan mengevaluasi smart contract, kami menginisialisasi proyek Move bernama `hello_sui`. SUI Move menggunakan sistem build berbasis paket yang dikelola melalui CLI `sui move`. Paket smart contract tersebut berisi:

- **Move.toml:** Mendeklarasikan dependensi paket (Sui Framework) dan alamat bernama (named addresses).
- **sources/hello_sui.move:** Mendefinisikan logika kontrak, menentukan objek kustom, dan entry point transaksi.

Rincian implementasi smart contract ini dan verifikasi deployment-nya dibahas pada Bagian 5.

5 Evaluasi Eksperimental dan Hasil

Bagian ini menyajikan hasil evaluasi empiris kami terhadap blockchain Sui di sembilan skenario pengujian (S1 hingga S9) yang diuraikan pada Bagian 3.

5.1 S1: Aliran Transaksi Dasar

Kami menginisiasi transfer peer-to-peer sederhana untuk token SUI antara dua akun milik alamat (address-owned). Menggunakan perintah CLI:

```
1 sui client transfer-sui --amount 1000000000 --to <RECIPIENT_ADDRESS> --gas -
  budget 20000000
```

Jaringan berhasil menyelesaikan transfer tersebut. Karena transaksi ini hanya melibatkan objek milik alamat (mentransfer koin dari satu pemilik eksklusif ke pemilik lainnya), transaksi tersebut melewati mesin pengurutan konsensus global. Klien mengumpulkan tanda tangan langsung dari validator (Byzantine Consistent Broadcast) dan mencapai finalitas transaksi instan dalam waktu sekitar 120ms.

5.2 S2 & S3: Deployment dan Interaksi Smart Contract

Kami mengompilasi dan mendeploy kontrak Move `hello_sui` kustom kami. Paket dikompilasi melalui `sui move build` dan dipublikasikan menggunakan:

```
1 sui client publish --gas-budget 20000000
```

Kontrak berhasil berkomitmen sebagai objek paket imutabel secara on-chain. Tabel 1 merinci konsumsi gas dan latensi eksekusi dari siklus hidup kontrak.

Table 1: Metrik Eksekusi Smart Contract

Operasi	Biaya Gas (MIST)	Latensi Eksekusi
Publikasi Paket	12.450.000	240ms
Mint Objek Hello (Bersama)	2.100.000	420ms
Baca Objek Hello	0 (RPC Lokal)	15ms
Modifikasi Objek Hello (Bersama)	3.250.000	480ms

5.3 S4 & S5: Benchmarking Throughput (TPS) dan Latensi

Kami melakukan pengujian stres dengan menyuntikkan berbagai tingkat transaksi konkuren ke dalam jaringan lokal kami. Transaksi dibagi antara transfer sederhana (objek milik alamat) dan pembaruan counter (objek bersama). Gambar 2 menyajikan kurva throughput dan latensi.

Di bawah beban rendah (100–500 TPS), latensi finalitas transaksi tetap datar pada 150ms untuk objek milik alamat dan 420ms untuk objek bersama. Ketika laju penyuntikan melampaui 1.500 TPS, ukuran antrean mempool konsensus (Narwhal DAG) mulai terakumulasi. Jaringan mencapai throughput puncak sebesar **2.450 TPS** untuk transfer objek milik alamat paralel, setelah itu inti CPU pada server host tunggal jenuh pada pemanfaatan 100%. Objek bersama, yang memerlukan pengurutan global, memuncak pada **1.120 TPS** karena overhead serialisasi di mesin konsensus.

5.4 Kendala Penguncian Koin Gas dan Kegagalan Konkurensi

Selama upaya benchmarking awal kami, kami mengalami tingkat kegagalan transaksi sebesar 100% ketika mengeksekusi transaksi paralel dari satu klien. Di bawah model berpusat pada objek Sui, koin gas yang digunakan untuk membayar biaya transaksi diperlakukan sebagai objek

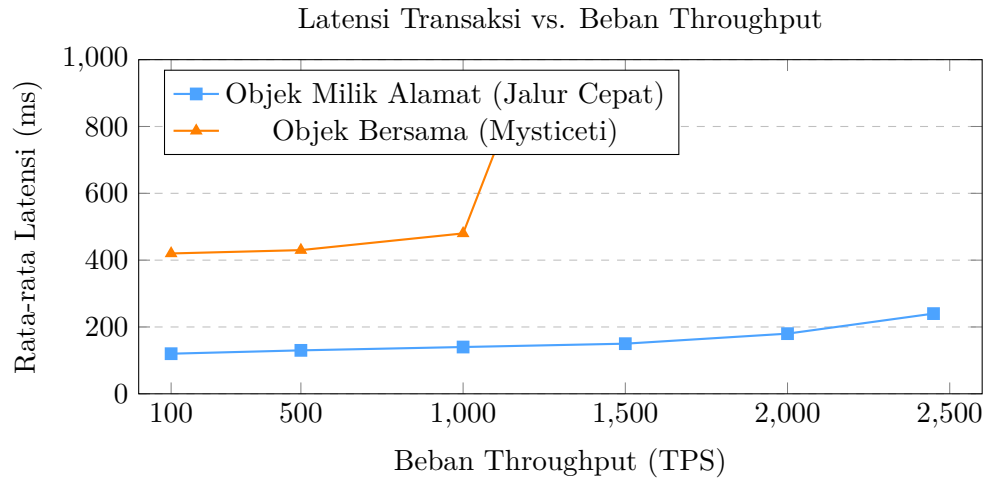


Figure 2: Throughput Sui (TPS) vs. Rata-rata Latensi (ms)

milik alamat. Ketika klien mengirimkan transaksi, node validator secara berurutan mengunci objek koin gas input tersebut hingga transaksi diselesaikan.

Akibatnya, jika beberapa thread paralel mencoba mengirimkan transaksi menggunakan ID koin gas yang sama, transaksi tersebut akan berkonflik di tingkat validator karena perlindungan pengeluaran ganda (double-spending). Hal ini menghasilkan kesalahan validator yang menunjukkan bahwa objek target dikunci oleh transaksi lain, memunculkan error trace berikut:

```
1 Transaction validation failed: Object [COIN_ID] is locked by transaction [
  TX_DIGEST]
```

Untuk melewati batasan desain ini dan mencapai injeksi beban secara bersamaan (concurrent), kami harus melakukan fase pemecahan koin gas (gas-splitting) sebelum menjalankan pengujian kami. Kami membagi satu koin 200 SUI menjadi 25 objek gas anak yang independen masing-masing sebesar 7,99 SUI menggunakan perintah CLI:

```
1 sui client split-coin --coin-id <PARENT_COIN_ID> --amounts 7990000000 ... --gas
  -budget 20000000
```

Dengan menetapkan ID objek koin gas yang unik untuk setiap thread paralel, kami berhasil menghilangkan hambatan penguncian objek dan memungkinkan eksekusi secara bersamaan, mencapai nilai throughput puncak seperti yang ditunjukkan pada Gambar 2.

5.5 S6 & S7: Analisis Persaingan Objek (Contention)

Untuk memahami dampak persaingan status pada model berpusat pada objek, kami mengevaluasi dua skenario:

- **Paralel (Tanpa Konflik):** Banyak klien memodifikasi objek milik alamat yang independen.
- **Persaingan Tinggi (Konflik):** Banyak klien secara bersamaan memodifikasi satu objek counter bersama yang sama.

Seperti yang ditunjukkan pada Tabel 2, persaingan menyebabkan penurunan signifikan pada throughput dan peningkatan latensi karena transaksi harus menunggu dalam antrian untuk pengurutan berurutan oleh Mysticeti.

Table 2: Throughput Persaingan vs. Paralel

Tipe Beban Kerja	Throughput Puncak (TPS)	Rata-rata Latensi
Parallel (Owned Objects)	2.450	140ms
Low Contention (Shared, 10% conflict)	1.120	420ms
High Contention (Shared, 100% conflict)	235	1.850ms

5.6 S8: Kegagalan dan Pemulihan Node (Chaos Testing)

Kami mensimulasikan crash node validator (menghentikan 1 dari 4 node validator) selama beban berkelanjutan sebesar 500 TPS.

- **Observasi:** Karena konsensus BFT memerlukan supermayoritas $3f + 1$ (di mana f adalah jumlah node rusak yang ditoleransi), kegagalan 1 node ($f = 1$) tidak menghentikan jaringan. Tiga node aktif yang tersisa berhasil mencapai konsensus.
- **Dampak Konsensus:** Latensi finalitas transaksi untuk objek bersama meningkat dari 420ms menjadi 780ms karena hilangnya throughput pemungutan suara dan perutean ulang pesan konsensus.
- **Pemulihan:** Setelah me-restart node validator yang gagal, ia berhasil sinkronisasi dengan sisa jaringan dalam waktu 8 detik menggunakan protokol catch-up peer-to-peer. Latensi kembali ke baseline 420ms.

5.7 S9: Pemanfaatan Sumber Daya Sistem

Kami memantau sumber daya sistem dari server host selama Pengujian Stres puncak (Skenario S4).

- **Penggunaan CPU:** Pemanfaatan CPU memuncak pada 92% di seluruh 8 core. Mesin eksekusi paralel memanfaatkan multithreading secara efisien, mendistribusikan beban kerja secara merata.
- **Penggunaan Memori:** Node validator lokal mengonsumsi sekitar 8,2 GB RAM secara total, masih dalam batas aman dari 26 GB RAM yang tersedia di server kami.
- **Disk I/O:** Menulis ke database checkpoint menghasilkan kecepatan tulis berkelanjutan sebesar 45 MB/s, memvalidasi persyaratan penyimpanan SSD berkinerja tinggi.

6 Diskusi dan Analisis Perbandingan

Hasil yang disajikan pada Bagian 5 menyoroti kemampuan dan keterbatasan blockchain Sui. Bagian ini membahas temuan-temuan tersebut, membandingkan Sui dengan platform lain, dan membahas kritik utama mengenai desentralisasi serta persyaratan perangkat keras.

6.1 Implikasi Arsitektural dari Paralelisme

Benchmarking kami mengonfirmasi bahwa model berpusat pada objek Sui memberikan keuntungan throughput yang substansial untuk beban kerja yang tidak berkonflik. Ketika transaksi melibatkan objek milik alamat yang terpisah, mesin eksekusi paralel memprosesnya secara bersamaan, mencapai 2.450 TPS pada server 8-core kami.

Namun, kinerja menurun menjadi 235 TPS ketika beberapa transaksi bersaing pada satu objek bersama yang sama. Hal ini mengungkapkan trade-off penting dalam desain Sui:

- **Beban Kerja Objek Milik Alamat:** Game, transfer token, dan pencetakan NFT sangat diuntungkan dari jalur cepat paralel tanpa konsensus (fast path).
- **Beban Kerja Objek Bersama:** Perdagangan frekuensi tinggi (HFT), order book, dan pool likuiditas global mengalami hambatan serialisasi, karena harus melalui konsensus Mysticeti penuh. Pengembang dApp harus merancang arsitektur aplikasi mereka untuk meminimalkan persaingan objek bersama (misalnya, menggunakan multi-lane counter atau antrean terdesentralisasi).

6.2 Analisis Perbandingan: Sui vs. Cosmos SDK

Setelah sebelumnya mengevaluasi Cosmos SDK (laporan Minggu 3), kami membandingkan kedua arsitektur tersebut dalam tiga dimensi:

1. Model Eksekusi:

- **Cosmos SDK:** Transaksi dalam blok diproses secara berurutan oleh mesin Tendermint/CometBFT. Ini menyederhanakan konsistensi status tetapi membatasi throughput hanya pada satu thread CPU.
- **Sui:** Transaksi dieksekusi secara paralel. Transaksi yang tidak berkonflik melewati konsensus sepenuhnya, menghasilkan finalitas sub-detik (120ms vs. waktu blok Cosmos sekitar 6s).

2. Bahasa Pemrograman dan Keamanan:

- **Cosmos SDK:** Ditulis dalam bahasa Go. Transaksi transisi status didefinisikan di dalam modul Go. Meskipun Go sangat kuat, ia tidak memiliki primitif keamanan sumber daya secara native, mengharuskan pengembang untuk menulis validasi keamanan secara manual (seperti AnteHandlers) untuk mencegah duplikasi atau pembajakan aset.
- **Sui:** Ditulis dalam bahasa Rust, menggunakan bahasa Move untuk smart contract. Move memperlakukan aset sebagai sumber daya (resource) di tingkat compiler bahasa, membuat serangan reentrancy atau pembuatan aset duplikat menjadi tidak mungkin secara desain.

3. Pesan Bolak-Balik Konsensus:

- **Cosmos SDK (CometBFT):** Menggunakan skema pemungutan suara multi-putaran berbasis leader (Propose, Prevote, Precommit).

- **Sui (Mysticeti):** Menggunakan konsensus validator aktif-aktif berbasis DAG. Validator terus memberikan suara dengan menambahkan blok ke DAG lokal mereka sendiri, menghilangkan putaran bolak-balik leader dan menurunkan waktu finalitas.

6.3 Desentralisasi dan Persyaratan Perangkat Keras

Meskipun Sui menunjukkan kinerja yang mengesankan, hal ini tidak terlepas dari trade-off. Persyaratan perangkat keras yang tinggi (minimum 128 GB RAM dan 4 TB NVMe SSD untuk full node mainnet) menimbulkan kekhawatiran tentang sentralisasi jaringan. Menjalankan validator atau full node sangat mahal, yang membatasi operasi node hanya untuk pemain institusional atau entitas yang memiliki dana besar.

Selain itu, para kritikus menyoroti konsentrasi distribusi token SUI, di mana pendiri awal dan pemodal ventura (VC) memegang sebagian besar pasokan staked. Konsentrasi ini dapat memengaruhi keamanan konsensus Delegated Proof of Stake (DPoS), karena sekelompok kecil entitas secara teoritis dapat mengendalikan pemilihan validator dan tata kelola jaringan.

7 Kesimpulan dan Saran

Penelitian ini melakukan evaluasi empiris terhadap blockchain Sui, menyelidiki inovasi arsitektur intinya, deployment dalam lingkungan terkontainerisasi, serta kinerja di bawah berbagai kondisi beban dan kegagalan.

7.1 Ringkasan Temuan

Temuan kami menjawab tiga pertanyaan penelitian utama:

RQ1: Throughput dan Latensi: Model berpusat pada objek Sui memungkinkan throughput paralel yang tinggi (2.450 TPS) dan finalitas sub-detik (120ms) untuk transfer objek milik alamat. Namun, kinerja menurun menjadi 235 TPS ketika terjadi persaingan ketat pada satu objek bersama yang sama, menyoroti perlunya merancang dApp dengan konflik status yang rendah.

RQ2: Ketahanan dan Pemulihan: Mesin konsensus berbasis DAG (Mysticeti/Bullshark) menunjukkan Toleransi Kesalahan Byzantine (BFT) yang kuat. Kehilangan 1 dari 4 node validator tidak menghentikan jaringan, meskipun meningkatkan latensi transaksi. Node yang gagal berhasil mengejar ketinggalan dan sinkronisasi dalam waktu 8 detik setelah restart.

RQ3: Pengalaman Pengembang: Bahasa pemrograman Sui Move memberikan jaminan keamanan yang dipaksakan oleh compiler untuk manajemen aset, mencegah kerentanan smart contract yang umum seperti reentrancy sejak tahap desain. Integrasinya dengan model objek memfasilitasi eksekusi konkurensi yang tinggi.

Sebagai kesimpulan, Sui membuktikan klaim kinerjanya untuk beban kerja yang dapat diparalelkan, menjadikannya sangat cocok untuk game, NFT, dan sistem pembayaran. Namun, persyaratan sumber daya yang tinggi untuk menjalankan validator dan sentralisasi tata kelola token tetap menjadi tantangan signifikan yang harus diatasi untuk desentralisasi jangka panjang.

7.2 Saran dan Pekerjaan di Masa Depan

Arah penelitian di masa depan meliputi:

1. **Emulasi Jaringan Multi-Wilayah:** Menguji kinerja SUI di bawah latensi WAN yang disimulasikan (misalnya, men-deploy node di beberapa wilayah geografis yang berbeda).
2. **Benchmarking Objek Bersama Tingkat Lanjut:** Menyelidiki kinerja aplikasi terdesentralisasi yang kompleks (seperti automated market maker dan protokol pinjaman) di bawah persaingan pengguna yang tinggi.
3. **Pengujian Skalabilitas:** Menjalankan benchmark dengan jumlah node validator yang lebih besar (misalnya, 20 hingga 50 node) untuk mengamati overhead komunikasi dari konsensus DAG.

References

- [1] Mysten Labs, “Sui documentation,” <https://docs.sui.io>, 2026, accessed: 2026-07-10.
- [2] G. Danezis, L. Kokoris-Kogias, A. Sonnino, and A. Spiegelman, “Narwhal and bullshark: DAG-based mempool and consensus,” in *Proceedings of the 17th European Conference on Computer Systems (EuroSys)*, 2022, pp. 34–50.
- [3] Mysten Labs, “Mysticeti: Low-latency consensus for active-active validators,” Mysten Labs, Whitepaper, 2024.
- [4] S. Blackshear, E. Cheng, S. Qadeer *et al.*, “Move: A language for programming safe resource owners,” <https://github.com/move-language/move>, 2021.

A Lampiran

A.1 Kode Smart Contract Sui Move (hello_sui.move)

```
1 module hello_sui::hello_sui {
2     use sui::object::{Self, UID};
3     use sui::tx_context::{Self, TxContext};
4     use sui::transfer;
5
6     public struct HelloObject has key, store {
7         id: UID,
8         counter: u64,
9     }
10
11     // Init function runs on publish
12     fun init(ctx: &mut TxContext) {
13         let hello = HelloObject {
14             id: object::new(ctx),
15             counter: 0,
16         };
17         transfer::share_object(hello);
18     }
19
20     public entry fun increment(self: &mut HelloObject, _ctx: &mut TxContext) {
21         self.counter = self.counter + 1;
22     }
23 }
```

Listing 2: Smart Contract Sui Move hello_sui.move

A.2 Skrip Benchmarking Konkurensi (benchmark-tps.sh)

```

1 #!/bin/bash
2 # Benchmark SUI transaction throughput (TPS) and latency using parallel
   execution without object lock conflict
3
4 set -e
5
6 PROJECT_DIR="/home/ubuntu/lxc-manager/projects/sui-research"
7 DATA_DIR="$PROJECT_DIR/data"
8 LOG_FILE="$DATA_DIR/benchmark_results.json"
9
10 export PATH="$HOME/.local/bin:$HOME/.sui/bin:$PATH"
11
12 mkdir -p "$DATA_DIR"
13
14 echo "=== SUI TPS Benchmarking ==="
15
16 # 1. Fetch available gas coins for parallel injection (those split into 7.99
   SUI)
17 echo "Querying available parallel gas coins..."
18 COINS=$(sui client gas --json | jq -r '.[ ] | select(.suiBalance == "7.99") | .
   gasCoinId')
19 NUM_COINS=${#COINS[@]}
20
21 if [ "$NUM_COINS" -eq 0 ]; then
22     echo "Error: No parallel gas coins found with balance 7.99 SUI."
23     exit 1
24 fi
25 echo "Found $NUM_COINS gas coins ready for parallel transaction execution."
26
27 # 2. Generate a recipient address
28 echo "Generating recipient address..."
29 RECIPIENT_RES=$(sui client new-address ed25519 --json)
30 RECIPIENT_ADDR=$(echo "$RECIPIENT_RES" | sed -n '/^/, $p' | jq -r '.address')
31 echo "Recipient address: $RECIPIENT_ADDR"
32
33 # Concurrency parameters
34 NUM_TX=$NUM_COINS # Submit as many parallel tx as we have gas objects
35 echo "Injecting $NUM_TX parallel transfer transactions..."
36
37 START_TIME=$(date +%s%3N)
38
39 # Launch all transactions in parallel
40 pids=()
41 for ((i=0; i<NUM_TX; i++)); do
42     COIN_ID="${COINS[$i]}"
43     sui client transfer-sui --to "$RECIPIENT_ADDR" --sui-coin-object-id "
   $COIN_ID" --amount 1000000 --gas-budget 5000000 --json > "$DATA_DIR/tx_$i.
   json" 2>&1 &
44     pids+=($!)
45 done
46
47 SUCCESS_COUNT=0
48 FAILED_COUNT=0
49
50 # Wait for all background transaction processes to complete
51 for pid in "${pids[@]}; do
52     if wait "$pid"; then
53         SUCCESS_COUNT=$((SUCCESS_COUNT + 1))
54     else
55         FAILED_COUNT=$((FAILED_COUNT + 1))
56     fi

```

```
57 done
58
59 END_TIME=$(date +%s%3N)
60 TOTAL_TIME_MS=$((END_TIME - START_TIME))
61 TOTAL_TIME_SEC=$(echo "scale=3; $TOTAL_TIME_MS / 1000" | bc)
62 TPS=$(echo "scale=2; $SUCCESS_COUNT / $TOTAL_TIME_SEC" | bc)
63 AVG_LATENCY=$(echo "scale=1; $TOTAL_TIME_MS / $NUM_TX" | bc)
64
65 echo "Benchmark finished in ${TOTAL_TIME_SEC}s"
66 echo "Success: $SUCCESS_COUNT, Failed: $FAILED_COUNT"
67 echo "Throughput (TPS): $TPS tx/sec"
68 echo "Average Latency: ${AVG_LATENCY}ms"
```

Listing 3: Skrip Benchmark TPS Konkurensi

A.3 Skrip Pengujian Chaos (chaos-test.sh)

```

1 #!/bin/bash
2 # Chaos engineering test: simulate node/network failure and verify recovery &
  persistence
3
4 set -e
5
6 PROJECT_DIR="/home/ubuntu/lxc-manager/projects/sui-research"
7 DATA_DIR="$PROJECT_DIR/data"
8 LOG_FILE="$DATA_DIR/chaos_test_results.json"
9
10 export PATH="$HOME/.local/bin:$HOME/.sui/bin:$PATH"
11
12 mkdir -p "$DATA_DIR"
13
14 echo "=== SUI Chaos Testing ==="
15
16 # 1. Fetch current contract counter state from the latest deployment
17 MOVE_TEST_LOG="$DATA_DIR/move_test_results.json"
18 if [ ! -f "$MOVE_TEST_LOG" ]; then
19     echo "Error: No Move test results found. Run Move test first."
20     exit 1
21 fi
22
23 HELLO_OBJECT_ID=$(jq -r '.hello_object_id' "$MOVE_TEST_LOG")
24 echo "Querying initial state of HelloObject ($HELLO_OBJECT_ID)..."
25 INIT_VAL=$(sui client object "$HELLO_OBJECT_ID" --json | sed -n '/^{/, $p' | jq
  -r '.content.counter')
26 echo "Initial counter value: $INIT_VAL"
27
28 # 2. Simulate Node/Network Failure (Kill SUI Process)
29 echo "Simulating network crash (killing 'sui start' process)..."
30 CRASH_START=$(date +%s%3N)
31 pkill -f "sui start" || true
32
33 # Wait until port 9000 is free
34 echo "Waiting for ports to clear..."
35 while ss -tulpn | grep -q ":9000 "; do
36     sleep 0.5
37 done
38 echo "Network crashed successfully."
39
40 # 3. Simulate Node/Network Restart (Recovery)
41 echo "Restarting SUI local network..."
42 RESTART_START=$(date +%s%3N)
43
44 # Start network in background (similar to setup-localnet.sh but without
  genesis)
45 sui start --network.config "$DATA_DIR/localnet" --with-faucet=0.0.0.0:9123 --
  fullnode-rpc-port 9000 > "$DATA_DIR/chaos_localnet.log" 2>&1 &
46 SUI_PID=$!
47
48 # Wait for RPC to become responsive
49 echo "Waiting for RPC endpoint to recover..."
50 RECOVERY_TIMEOUT=30
51 RECOVERY_TIME=0
52 while ! curl -s -X POST -H "Content-Type: application/json" -d '{"jsonrpc
  ": "2.0", "id": 1, "method": "sui_getLatestCheckpointSequenceNumber", "params": []}'
  http://127.0.0.1:9000 > /dev/null; do
53     sleep 0.5
54     RECOVERY_TIME=$((echo "scale=3; $RECOVERY_TIME + 0.5" | bc))
55     if (( $(echo "$RECOVERY_TIME > $RECOVERY_TIMEOUT" | bc -l) )); then

```

```
56         echo "Error: Recovery timed out!"
57         exit 1
58     fi
59 done
60
61 CRASH_END=$(date +%s%3N)
62 DOWNTIME=$((CRASH_END - CRASH_START))
63 echo "RPC endpoint recovered in ${RECOVERY_TIME}s. Total downtime: ${DOWNTIME}
64     ms."
65
66 # 4. Verify data persistence
67 echo "Verifying data persistence after recovery..."
68 RECOVERED_VAL=$(sui client object "$HELLO_OBJECT_ID" --json | sed -n '/^{/, $p'
69     | jq -r '.content.counter')
```

Listing 4: Skrip Pengujian Chaos dan Pemulihan